

Tree Dash Notes

Notes on various observations required in order to solve **Tree Dash**.

Ethan Cai

Contents

1	Notation	2
1.1	Lemma 1	2
1.2	Lemma 1.2	2
1.3	Lemma 2	2
1.4	Lemma 3	3
1.5	Lemma 4	3
1.6	Lemma 5	3
1.7	Lemma 6	3
1.8	Lemma 7	4
1.9	Lemma 8	4
1.10	Lemma 9	4
2	Solution	4

§ 1 Notation

Definition 1.1. We define the minimum and maximum ancestors to be $\min \text{anc}(u)$ and $\max \text{anc}(u)$ respectively.

Similarly we define the minimum and maximum descendants to be $\min \text{desc}(u)$ and $\max \text{desc}(u)$.

We also define the dependencies of a node u to be the set of all nodes such that u is reachable from that node. In that vein, the dependency sum is the sum of weights of all dependencies of a node (including itself).

We call the tree from the input data to be the **initial tree**, the graph created from the various operations to be the **initial graph** and the eventual condensation graph to be the **SCC graph**.

§1.1 Lemma 1

Claim. You can reduce the two ancestor edges into a single **up edge**.

Proof. For any node u , consider the two orderings of the min and max ancestor edge destinations.

One of the following is true: Either $\min \text{anc } u \in \text{Anc}(\max \text{anc } u)$ or $\max \text{anc } u \in \text{Anc}(\min \text{anc } u)$. In English, one node must be above the other.

If so, that means you can reach the higher node from the lower node.

Since the lower node can reach anything the higher node can, there is no reason to keep both edges.

We will henceforth refer to this compressed edge as the **up edge** and denote it $\text{up } u$.

§1.2 Lemma 1.2

Claim. For any two nodes a and b , if there exists an edge $a \rightarrow b$, either $\text{up } b = a$ or $\text{up } b = \text{up } a$.

Proof. Trivial. If a was a new max/min along b 's ancestor path, b 's up-edge would go to a . Otherwise, they must share the same up-edge.

§1.3 Lemma 2

After running SCC, let's call any SCC that only has one node a **singleton** and any SCC with more than one node **real**.

Definition 1.2. For a given real SCC S , we define the **top** of S to be the node in S whose depth is the lowest. We denote this $\text{top } S$.

Claim. There are no edges from real SCC's to singletons.

Proof. Proof by contradiction.

Let the real SCC be S and the singleton be u . Assume for the sake of contradiction that there existed an edge $S \rightarrow u$.

If u was below S , clearly S is an ancestor of u . However, since S is a real SCC, there must exist some element $x \in S$ such that $\text{up } x = \text{top } S$. Therefore S must be a new minimum or maximum along u 's ancestor path. Hence, u must also be connected to S . However, we defined u to be a singleton. Contradiction.

A similar argument can be used to disprove the case where u is above S .

§1.4 Lemma 3

With the same logic as [Lemma 2](#), it can be shown that all real SCC's cannot have a downwards edge.

§1.5 Lemma 4

Claim. The real SCC's in the SCC graph can be reduced to form a tree while keeping connectivity.

Proof. Take an SCC S . It cannot have an edge to anything below it because of [Lemma 3](#). However, you can repeatedly take up-edges until it hits the root.

Let another real SCC T be reachable from S . By properties of the initial tree, anything T can reach can also be reached from S . Observe that you only need the deepest real SCC reachable from S to preserve connectivity. Since every node is connected and there is one retained up-edge per real SCC (except for the root), these connections in the SCC graph can be reduced to a tree while keeping connectivity.

Since the SCC graph forms a tree, we will call it the SCC tree. The path sum of a SCC S is the sum of all weights of all SCC's on the path from S to the root SCC.

§1.6 Lemma 5

We define the singleton-graph to be the subset of the SCC graph with only singleton nodes.

Claim. The order of dependency in the singleton-graph is the same as the depth order in the initial tree.

Proof. Trivial. Consider a singleton u . It has an up-edge and down-edge(s). If the down-edge led to above u , it ceases to be a down-edge.

Suppose there was somehow a case where $u \xrightarrow{\text{up-edge}} v \xrightarrow{\text{down-edge}} w$, breaking the depth order. w must also have an edge back to v because v is a new min/max along its ancestor path. This creates a real SCC consisting of v and w , contradiction.

§1.7 Lemma 6

Define the **up-component** of a singleton u to be the first real SCC that is reachable by repeatedly taking up-edges from u .

Claim. If a singleton u has an up-component S , all of u 's descendants must also have up-component S .

Proof. Let u be a singleton and v be one of u 's children. Since u has an up-edge, v must either have the same up-edge, or point to u by [Lemma 1.2](#). Thus, they must reach the same up-component. Claim for all descendants can be proven by induction.

§1.8 Lemma 7

Claim. Let u be a singleton. The set X of its dependencies must satisfy $X \subseteq \text{Anc } u$.

Proof. Trivial. If a dependency v was not in $\text{Anc } u$, there could not have been an edge from v to u .

§1.9 Lemma 8

You can prevent double-counting of ancestors in the singleton-graph by compressing down-edges.

Consider the case $a \rightarrow b \rightarrow u$, $a \rightarrow u$. Here, naively computing the the dependency sum would double-count a . However, we can observe that we can remove the edge $a \rightarrow u$, as it is already handled by $a \rightarrow b \rightarrow u$.

Whenever a node u has two down-edges, call them a and b . If $a \in \text{Anc } b$ there must be an edge from $u \rightarrow a$ and $u \rightarrow b$, therefore we can remove $u \rightarrow b$ and keep $u \rightarrow a \rightarrow b$.

This transformation turns the singleton-graph into this weird hybrid directed tree thingy. This in turn also makes calculating the dependency sum for singleton nodes trivial.

§1.10 Lemma 9

Claim. If a singleton u has a down-edge that leads to a real SCC, that real SCC must have the same up-component as u .

Proof. Let the destination of the down-edges be A . A must not connect to u by [Lemma 2](#). However, that means that A and u must share an up-edge, so they must all have the same up-component.

§ 2 Solution

- Compress up-edges and down-edges
- Run SCC
 - Compute SCC tree by sorting by `top S`
- Compute path sums for all SCC's in SCC tree.
- Go by reverse postorder through all singleton nodes:
 - Let $\text{wr}[u]$ denote the sum of weights of the real SCC's that are reachable from u . Let

$$\text{weight } u = \begin{cases} \text{wr}[u] & \text{if } u \text{ is a singleton} \\ \text{path sum of } u & \text{if } u \text{ is a real SCC} \end{cases}$$

- Let **base** = path sum of u
- If u has no children, $\mathbf{wr}[u] = \mathbf{base}$
- If u has one child a , $\mathbf{wr}[u] = \text{weight } a$
- If u has two children a and b , $\mathbf{wr}[u] = \text{weight } a + \text{weight } b - \mathbf{base}$.
 - Why? If you have one child, you do not want to add **base** since you would double count it.
 - If you have two children, you double-count **base** once, so you need to subtract it to compensate. This reasoning follows for real SCC's by **Lemma 9**. If you had two real SCC's A and B from the two down edges, they must both overlap **base** since they share the up-edge. Therefore you **still** have to subtract it. Similar reasoning applies for one real SCC from down-edges.
- Let $\mathbf{ws}[u]$ denote the sum of weights of singletons that are reachable from u . This is trivially obtainable as a “tree” dp.

The answer is:

$$\begin{aligned} \text{ans} = & \sum_{\text{singleton } u} W_u \cdot (\mathbf{ws}[u] + \mathbf{wr}[u] - W_u) \\ & + \sum_{\text{real SCC } S} \left(\text{path sum of } \cdot \sum_{u \in S} W_u \right) - \sum_{u \in S} W_u^2 \end{aligned}$$

All of this can be implemented in $O(N)$.